关于期中考试系统中需要提交程序的说明

- 其他题目：提交整个函数（包括函数头）

```
def plus(a,b):
    return a+b
```

  - 结果函数返回，不要只print

- 面向对象题目：提交基类、继承的类（以及调用函数，可选）
  - 你需要去补充完整基类和继承类的内容

考试须知

- 时间地点：周一晚，18:30-20:30 共10道题，上机机房
- 需要携带：**身份证明 （学生卡）**
- 允许携带：笔、适量的衣物、食物、饮用水
- 不许携带：书本、任何电子设备、存储设备、通讯设备和机器外设，例如个人电脑、U 盘、手机和键盘
- 允许使用PyCharm Python开发环境编写和测试你的程序。
  - 同时，考试时可以访 问上述开发环境自带的教程和文档
- 评测采取黑盒测试
  - 每道题目会准备若干组测试数据，对于每组数据分别运行你的程序并检查是否在限制的时间、空间中正确运行得到答案，机器会根据你正确的数据组数给出这道题目的得分。
  - 机器故障可以举手请求工作人员修复
    - 非选手错误操作的故障，耽误时间超过 3 分钟的，可以请求工作 人员延长个人提交时间。自行处理故障的不能申请延长提交时间。
  - 除非题目允许，你的程序只能使用Python3自带的函数和数据类型以及自带的库
  - 每道题目可以进行 32 次有效提交
- 现场实际情况与上述有区别的，以现场通知为准。

A_实现函数sort_odd_and_even

题目：实现函数sort_odd_and_even

刷新

实现sort_odd_and_even函数。如果输入不是一个列表或者输入列表的元素不全是整数，返回None； 如果输入列表的元素均为整数，首先将奇数从大到小排序，其后将偶数按照从小到大排序；如果输入为空列表，返回空列表。

```
def sort_odd_and_even(a):
    # 补全该函数
```

例如：

调用： sort_odd_and_even([1.3, 2, 3])

返回： None

调用： sort_odd_and_even([1, 2, 3, 4, 5])

返回： [5, 3, 1, 2, 4]

调用： sort_odd_and_even([1, 3, 5, 7])

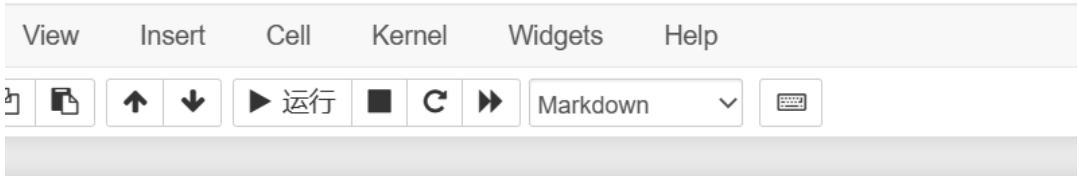
返回： [7, 5, 3, 1]

调用： sort_odd_and_even([])

返回： []

```python
def sort_odd_and_even(a):
    if type(a) == list:
        for i in a:
            if type(i) != int:
                return None
        odd, even = [], []
        for i in a:
            odd.append(i) if i % 2 == 1 else even.append(i)
        odd.sort(reverse=True)
        even.sort()
        return odd + even
    else:
        return None
```

B_实现密码合法性检查 (已自动保存)



题目：实现密码合法性检查

输入一个字符串，检测是否是合法的密码：

1）密码必须是一个字符串

2）密码必须包含6~12个字符

3）密码至少包含一个大写字母、一个小写字母和一个数字

4）密码至少包含三个特殊字符"$#@"中的一个

返回： 如果是一个合法的密码，返回True，否则返回False

```
def password_check(a):
    # 补全该函数
```

例如：

调用： password_check("a1b2c3d4e5")

返回： False

调用： password_check("a1b2c3d4E@")

返回： True

调用： password_check(123)

返回： False

```python
def password_check(a):
    if type(a) != str or len(a) < 6 or len(a) > 12:
        return False
    small, cap, num, nonletter = False, False, False, False

    for p in a:
        if p in "abcdefghijklmnopqrstuvwxyz":
            small = True
        elif p in "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
            cap = True
        elif p in "0123456789":
            num = True
        elif p in "$#@":
            nonletter = True
        else:
            pass

    return small and cap and num and nonletter
```

题目：实现函数recursive_digit_sum

实现recursive_digit_sum函数，该函数的功能如下，首先计算输入的非负整数的各位数字之和，如果该和大于9，则重复上述过程，即计算和的各位数字之和，直至求出的和是一个一位数为止，并输出最后的结果

```
def recursive_digit_sum(a):
    # 补全该函数
```

例如：

调用： recursive_digit_sum(32)

返回： 5

说明：3 + 2 = 5

调用： recursive_digit_sum(678)

返回： 3

说明：先计算6 + 7 + 8 = 21，再计算2 + 1 = 3

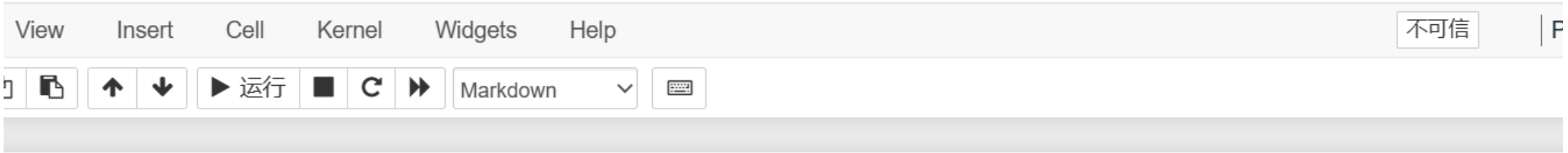
调用： recursive_digit_sum(88888888888888)

返回： 4

说明：输入中共有14个8，求和为8 * 14 = 112，再计算1 + 1 + 2 = 4

```
def recursive_digit_sum(a):
    if a < 10:
        return a
    else:
        b = sum([int(x) for x in str(a)])
        return recursive_digit_sum(b)
```

D_reverse_words (更改未保存)



题目：实现函数reverse_words

实现reverse_words函数，将输入字符串中的每个单词都逆序后输出，注意单词的顺序不能改变，单词之间用空格分隔，字符串中没有标点符号

```
def reverse_words(a):
    # 补全该函数
```

例如：

调用： reverse_words('Hello World')

返回： olleH dlroW

调用： reverse_words('python package')

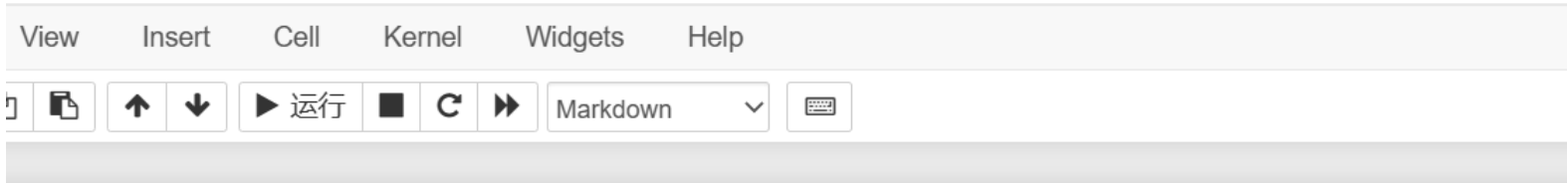
返回： nohtyp egakcap

调用： reverse_words('Renmin University of China')

返回： nimneR ytisrevinU fo anihC

```python
def reverse_words(a):
    l = a.split(' ')
    for i in range(len(l)):
        l[i] = l[i][::-1]
    print(' '.join(l))
```

E_整除is_ divisible (更改未保存)



题目：实现函数is_divisible

实现is_divisible函数。如果输入是一个整数，检查该整数能否被其所有数字之和整除；如果输入不是整数，返回"None"。

```
def is_divisible(a):
    # 补全该函数
```

例如：

调用： is_divisible(123)

返回： No

调用： is_divisible("-1.0f")

返回： None

调用： is_divisible(-102)

返回： Yes

调用： is_divisible(10)

返回： Yes

```
def is_divisible(a):
    if type(a) != int:
        return 'None'

    b = str(a)
    if b[0] == '-':
        b = b[1:]
    c = sum([int(x) for x in b])

    if c == 0:
        return 'Yes'
    elif a % c == 0:
        return 'Yes'
    else:
        return 'No'
```

题目：实现函数max_in_list

实现max_in_list函数。输入一个数组，计算数组中所有出现过的*整数*或者*浮点数*的最大值；数组中元素若为组合数据类型，只可能是元组、列表或集合；若数组中不包括任何*整数*和*浮点数*，则返回None。

```
def max_in_list(data):
    # 补全该函数
```

例如：

调用： max_in_list([10,-0.2])

返回： 10

调用： max_in_list(["15.5", 7, [(9,2),-2]])

返回： 9

调用： max_in_list(['a',('b','c')])

返回： None

调用： max_in_list([5, {16, 2}])

返回： 16

```python
def max_in_list(data):
    max_value = None #-1e64
    if isinstance(data, float) or isinstance(data, int):
        return data
    elif isinstance(data, list) or isinstance(data, set) or isinstance(data, tuple):
        for i in data:
            temp_value = max_in_list(i)
            if temp_value is not None:
                if max_value is None or temp_value>max_value:
                    max_value = temp_value
    return max_value
```

题目：实现函数min_in_list_2

实现min_in_list_2函数。输入一个数组，计算数组中所有出现过的*整数*或者*浮点数*中第二小的值以及该值对应的索引。数组中元素只可能是*整数*、*浮点数*或者*字符串*；若数组中包括的*整数*和*浮点数*的个数小于2，则返回None；若数组中第二小的数出现多次，则返回第一次出现时的索引；若第二小的数和最小的数相等，则顺序在前的为最小的数。

```
def min_in_list_2(data):
    # 补全该函数
```

例如：

调用： min_in_list_2([10,-0.2])

返回： (10, 0)

调用： min_in_list_2(["15.5", 7, 9, 2, -2, 2])

返回： (2, 3)

调用： min_in_list_2(['a','b','c'])

返回： None

调用： min_in_list_2([5, 2, 2])

返回： (2, 2)

```python
def min_in_list_2(data):
    data2 = []
    for i in range(len(data)):
        if isinstance(data[i],int) or isinstance(data[i],float):
            data2.append((data[i],i))
    data2.sort()
    if len(data2)<2:
        return None
    else:
        return data2[1]
```

实现word_cut(string, dictionary)函数。将字符串string分割为若干个dictionary中包含的词。

注意同一个字符串可能有不同的分词方式，我们需要选择其中最优的一种。因此，dictionary为一个字典，该字典的键为合法的词，对应的值为该词的得分(一个*正整数*) 。一种分词方式的得分即为所有词的得分的和，最优的分词方式为得分和最大的分词方式。

word_cut函数需要返回一个包含最优分词结果(list)和最优分词结果的得分的元组。若输入为空字符串，应返回([], 0);若字符串不能被分割为字典中的词，应返回(None, 0)。

```
def word_cut(string, dictionary):
    # 补全该函数
```

例如：

调用：

```
word_cut("abcabc", {"a": 1, "b": 1, "c": 1})
```

返回：

```
(['a', 'b', 'c', 'a', 'b', 'c'], 6)
```

解释：分成六个字母，每个字母的得分均为1，所以总得分为6。

调用：

```
word_cut("abcabd", {"a": 1, "b": 1, "c": 1})
```

返回：

```
(None, 0)
```

解释：d不在字典中，没有合法的分词方式，返回(None, 0)

调用：

```
word_cut("南京市长江大桥", {"南京": 10, "南京市": 12, "市长": 10, "长江": 10, "大桥": 10, "长江大桥": 25, "江大桥": 1})
```

返回：

```
(['南京市', '长江大桥'], 37)
```

解释：该字符串有歧义，因此有多种分词方式，如['南京市', '长江', '大桥']（得分为12+10+10=32）和['南京', '市长', '江大桥']（得分为10+10+1=21)。我们需要返回其中最优的一种['南京市', '长江大桥']（得分为12+25=37）。

调用：

```
word_cut("中华人民共和国", {"中华": 1, "人民": 1, "共和国": 1, "中华人民": 1, "中华人民共和国": 1})
```

返回：

```
(['中华', '人民', '共和国'], 3)
```

解释：若每个词的得分均相等，分词函数会倾向于把词切得更细。

调用：

```
word_cut("中华人民共和国", {"中华": 10, "人民": 10, "共和国": 15, "中华人民": 22, "中华人民共和国": 100})
```

返回：

```
(['中华人民共和国'], 100)
```

```python
# 不使用记忆内的程序
def word_cut(string, dictionary, memory={}):
    if len(string) == 0:
        return [], 0
    else:
        best_word_list = None
        best_score = 0
        for i in range(1, len(string)+1):
            word = string[0:i]
            cur_score = dictionary.get(word, -1)
            if cur_score == -1:
                continue

            word_list, score = word_cut(string[i:], dictionary)
            if word_list is not None and cur_score + score > best_score:
                best_word_list = [word] + word_list
                best_score = cur_score + score

        return best_word_list, best_score
```

```python
def word_cut(string, dictionary, memory={}):
    if len(string) == 0:
        return [], 0
    elif string in memory:
        return memory[string]
    else:
        best_word_list = None
        best_score = 0
        for i in range(1, len(string)+1):
            word = string[0:i]
            cur_score = dictionary.get(word, -1)
            if cur_score == -1:
                continue

            word_list, score = word_cut(string[i:], dictionary)
            if word_list is not None and cur_score + score > best_score:
                best_word_list = [word] + word_list
                best_score = cur_score + score

        memory[string] = (best_word_list, best_score)
        return best_word_list, best_score
```

题目：奇怪的字典序

在英文字典中，排列单词的顺序是先按照第一个字母以升序排列（即a、b、c……z 的顺序）；如果第一个字母一样，那么比较第二个、第三个乃至后面的字母。如果比到最后两个单词不一样长（比如，sigh 和 sight），那么把短者排在前。我们把这种排序方式称为字典序。然而，现在有一本奇怪的字典，其中字母的顺序不是a-z，而是由一个包含26个小写字母的字符串给定的。

请实现sorted_with_weird_order(string_list, order)函数。将string_list中包含的字符串（字符串中只包含小写字母）按照order中字母顺序定义的字典序进行排序。

```
def sorted_with_weird_order(string_list, order):
    # 补全该函数
```

例如：

调用：

```
sorted_with_weird_order(['c', 'b', 'a'], 'abcdefghijklmnopqrstuvwxyz')
```

返回：

```
['a', 'b', 'c']
```

解释： order中字母顺序为a-z，排序结果和普通字典序一样。

调用：

```
(['', 'ab', 'a'], 'abcdefghijklmnopqrstuvwxyz')
```

返回：

```
['', 'a', 'ab']
```

解释： 字典序中若前缀一样，较短的词会排在靠前的位置。

调用：

```
sorted_with_weird_order(['c', 'b', 'a'], 'acbdefghijklmnopqrstuvwxyz')
```

返回：

```
['a', 'c', 'b']
```

解释： order中abc三个字母的顺序改变了，排序结果相应的也发生了改变。

数据范围：

0 <= len(string_list) <= 100000

对于string_list中的任意元素string：

0 <= len(string) <= 1000

```
def sorted_with_weird_order(string_list, order):
    d = {c:i for i, c in enumerate(order)}
    return sorted(string_list, key=lambda string: [d[c] for c in string])
```

```
languages = ['Python', 'Java', 'JavaScript']
enumerate_prime = enumerate(languages)
# convert enumerate object to list

print(list(enumerate_prime))
# Output: [(0, 'Python'), (1, 'Java'), (2, 'JavaScript')]
```

r J_面对对象实现宠物店管理 (更改未保存)

Logout

View Insert Cell Kernel Widgets Help 不可信 Python 3 (ipykernel)

运行 Markdown

采用面向对象方法实现宠物店管理：

有一个父类 `Animal`，具有属性 `name` 和方法 `speak()`。其中，`speak()` 方法打印出 `"I am an animal."`

有两个子类：`Dog` 和 `Cat`。它们继承自 `Animal` 类，并且都具有一个额外的方法 `play()`。其中，`Dog` 类的 `speak()` 方法输出 `"I am a Dog."`，`play()` 方法输出 `"Dog {NAME} is playing."`；`Cat` 类似。

现在，有一个 `PetShop` 类，管理所有宠物的实例。该类具有以下方法：

- `add_pet(pet:Animal)`：将一个宠物实例添加到宠物店中。
- `show_pets()`：显示所有添加到宠物店中的宠物的名称和类型（狗或猫）。

要求：

请你填写如下 Python 代码，采用面向对象方法，使其能够顺利运行，满足上述要求。

```
class Animal(object):
    def __init__(self, name):
        self.__name = name

    def speak(self):
        print("I am an animal.")

class Dog(Animal):
    def __init__(self, name):
        super(Dog, self).__init__(name)
        self.__species = 'Dog'

    def speak(self):
        # 补全该函数

    def play(self):
        # 补全该函数

class Cat(Animal):
    def __init__(self, name):
```

我们会使用以下函数，测试你编写的类是否能满足要求（注意：你无法修改改测试函数）：

```
def testing_PetShop(pet_dict):
    pet_shop = PetShop()
    for species, name in pet_dict.items():
        species = species.split("_")
        if species[0] == "Dog":
            dog = Dog(name)
            dog.speak()
            dog.play()
            pet_shop.add_pet(dog)
        elif species[0] == "Cat":
            cat = Cat(name)
            cat.speak()
            cat.play()
            pet_shop.add_pet(cat)
        else:
            print("Incorrect dict keys!")
    pet_shop.show_pets()
```

使用样例：

```
testing_PetShop({"Dog_1":"Tom", "Dog_2":"Bob", "Cat":"Lucy"})
```

输出结果：

```
I am a Dog.
Dog Tom is playing.
I am a Dog.
Dog Bob is playing.
I am a Cat.
Cat Lucy is playing.
Tom is Dog.
Bob is Dog.
Lucy is Cat.
```

```python
class Animal(object):
    def __init__(self, name):
        self.name = name

    def speak(self):
        print("I am an animal.")

class Dog(Animal):
    def __init__(self, name):
        super(Dog, self).__init__(name)
        self.__species = 'Dog'

    def speak(self):
        print("I am a Dog.")

    def getname(self):
        return self.name

    def play(self):
        print("Dog {0} is playing.".format(self.name))

class Cat(Animal):
    def __init__(self, name):
        super(Cat, self).__init__(name)
        self.__species = 'Cat'

    def speak(self):
        print("I am a Cat.")

    def getname(self):
        return self.name

    def play(self):
        print("Cat {0} is playing.".format(self.name))

class PetShop(object):
    def __init__(self):
        self.pets = []

    def add_pet(self, pet):
        self.pets.append(pet)

    def show_pets(self):
        for pet in self.pets:
            if isinstance(pet,Dog):
                print("{} is Dog.".format(pet.getname()))
            else:
                print("{} is Cat.".format(pet.getname()))
```